

All or Nothing: How Library Type Annotations Affect Client-Side Errors in Python



ERIC ASARE

جامعة نيويورك أبوظبي
NYU ABU DHABI



LUCA DI GRAZIA

University of St.Gallen



SARAH NADI

جامعة نيويورك أبوظبي
NYU ABU DHABI

What are type annotations?

No type annotation

```
def naturaldelta(
    value,
    months,
    minimum_unit,
):
```

Has type annotation

```
def naturaldelta(
    value: dt.timedelta | float,
    months: bool,
    minimum_unit: str,
) -> str:
```

Python type annotation indicates the type of data a variable is expected to hold.

The promised benefits of type annotations

Python annotations promise to reduce type-related errors



Library (Humanize)

```python
def naturaldelta(
    value: dt.timedelta | float,
    months: bool,
    minimum_unit: str,
) -> str:
```

```python
def _private_api(
    value: float,
    ) -> int:
```



Client

```python
from humanize import naturaldelta

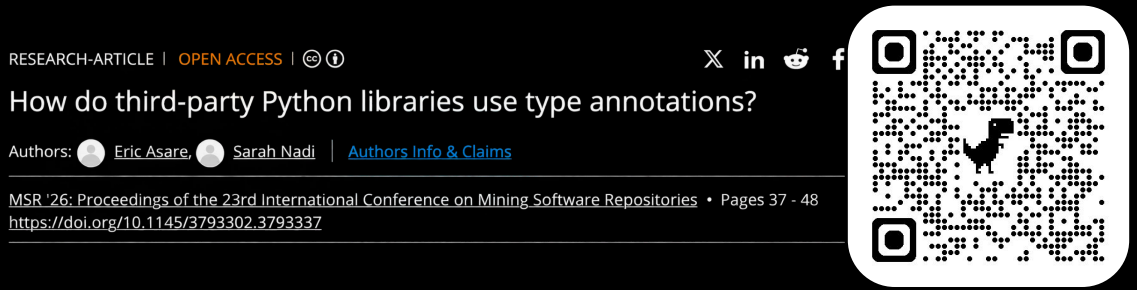
data = "1000001"
human_format = naturaldelta(data)
```

PYRIGHT

Argument of type "str" cannot be assigned to parameter of type "timedelta | float"
- pyright

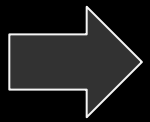
Do Python libraries annotate their public APIs?

YES! 83% of popular Python libraries annotate their public APIs. The median coverage is 20%.



Given that 83% of popular Python libraries have adopted type annotations …

Client projects using these libraries in their code



have less type errors

Our key question: In practice, do we see less type errors related to third-party libraries?

Our Dataset

SEART GitHub

13,751

Active Software Engineering Projects (Client Projects)

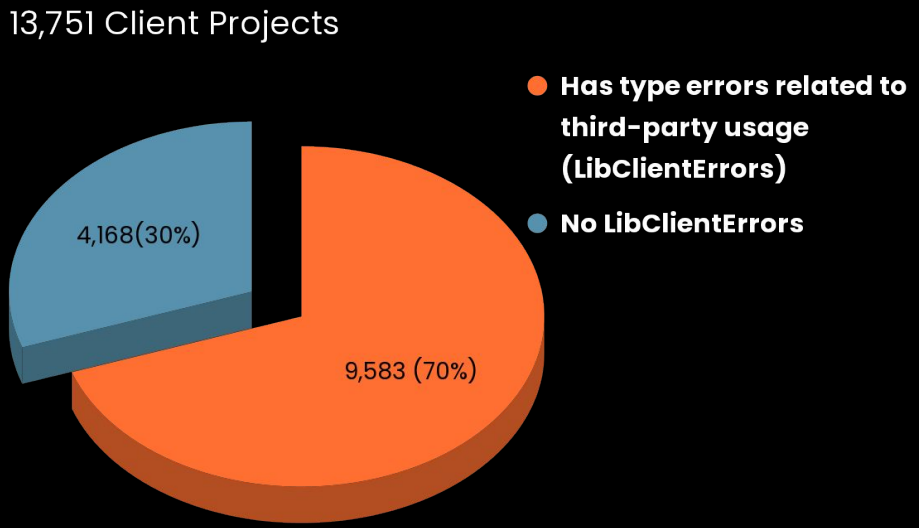


28,333

Third Party Libraries

Our Hope: These clients should not have errors related to their libraries' APIs **(LibClientError)**

RQ1: Do client projects make type errors involving third-party libraries?



Finding 1: 70% of client projects contain at least one LibClientError

It seems that the promise of type annotations does not hold in practice

Because 70% of client projects contain at least one LibClientError,

Maybe it's just a few libraries that are causing these errors

RQ2: Are certain libraries problematic?

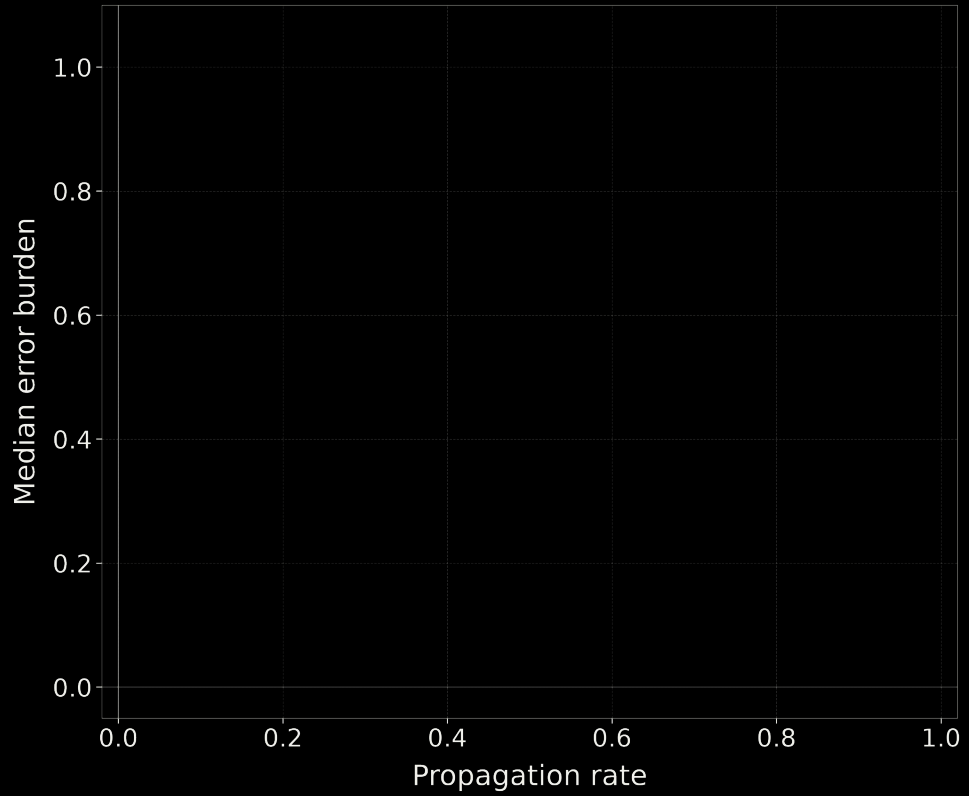
Library:
Humanize



(1) **Propagation Rate** (PropRate) = 2/3 = 67%

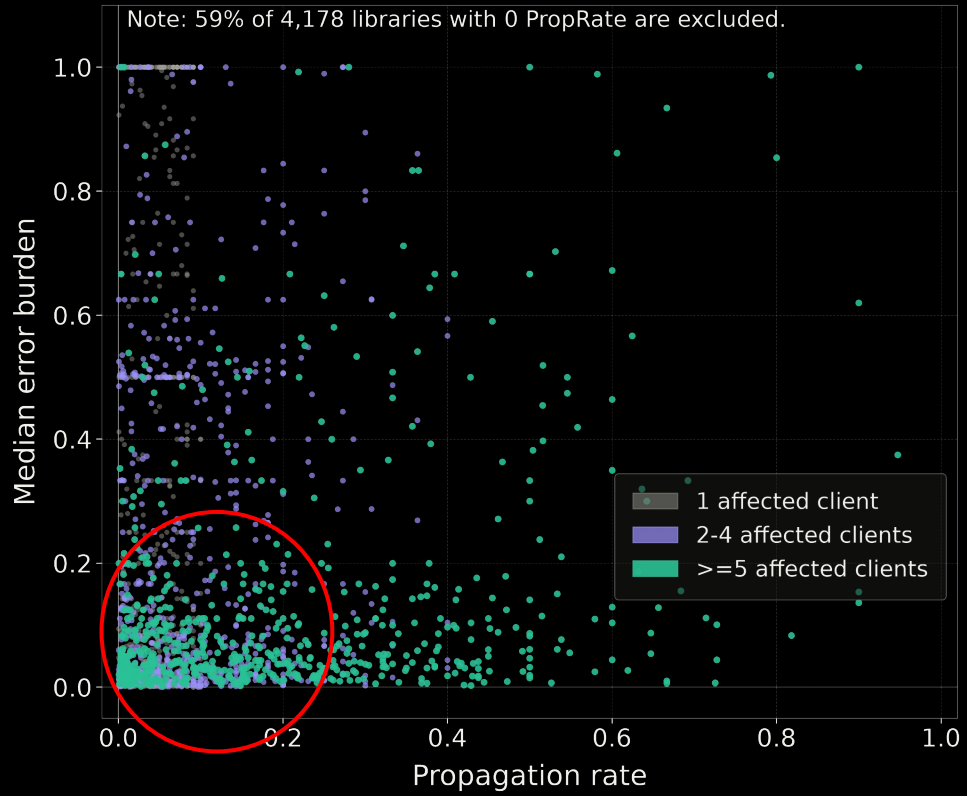
(2)**Median Error Burden** (ErrBurden) = (30% and 50%= 40%)

RQ2: Are certain libraries problematic?



RQ2: Are certain libraries problematic?

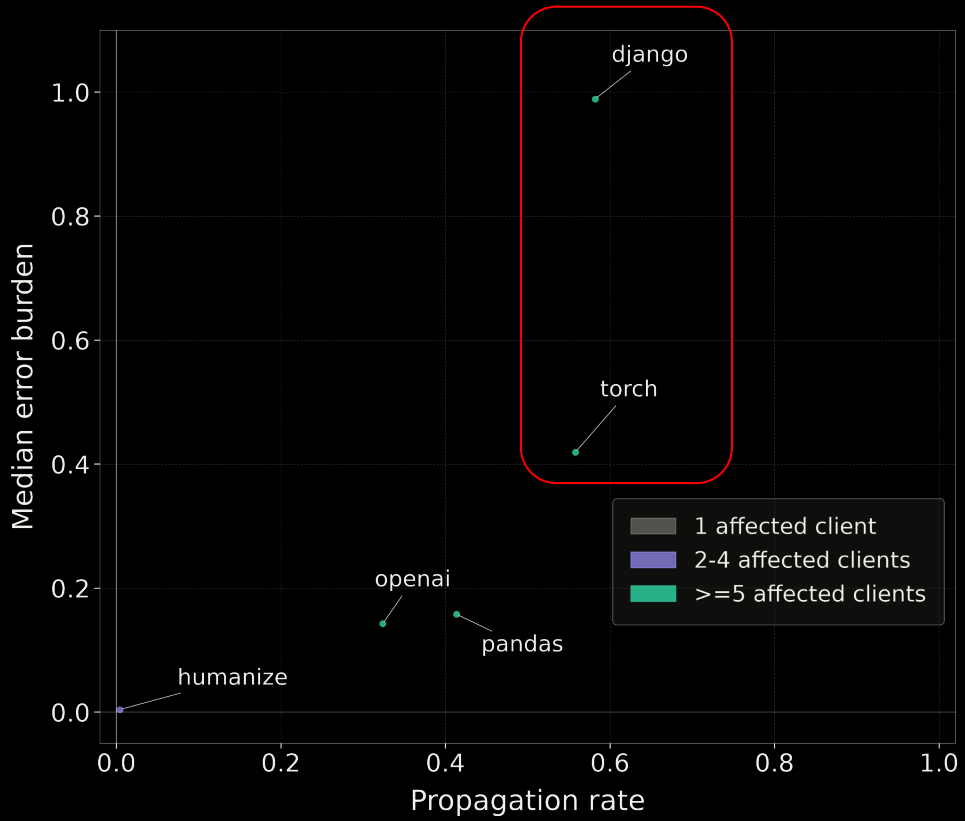
The distribution is uneven across libraries.



RQ2: Are certain libraries problematic?

For example, django and torch have similar propagation rates, but django has a much higher median error burden.

Finding 2: Some libraries are indeed more problematic than others, causing more errors in the ecosystem



So far , In practice, the state of type errors isn't matching the promise of annotations

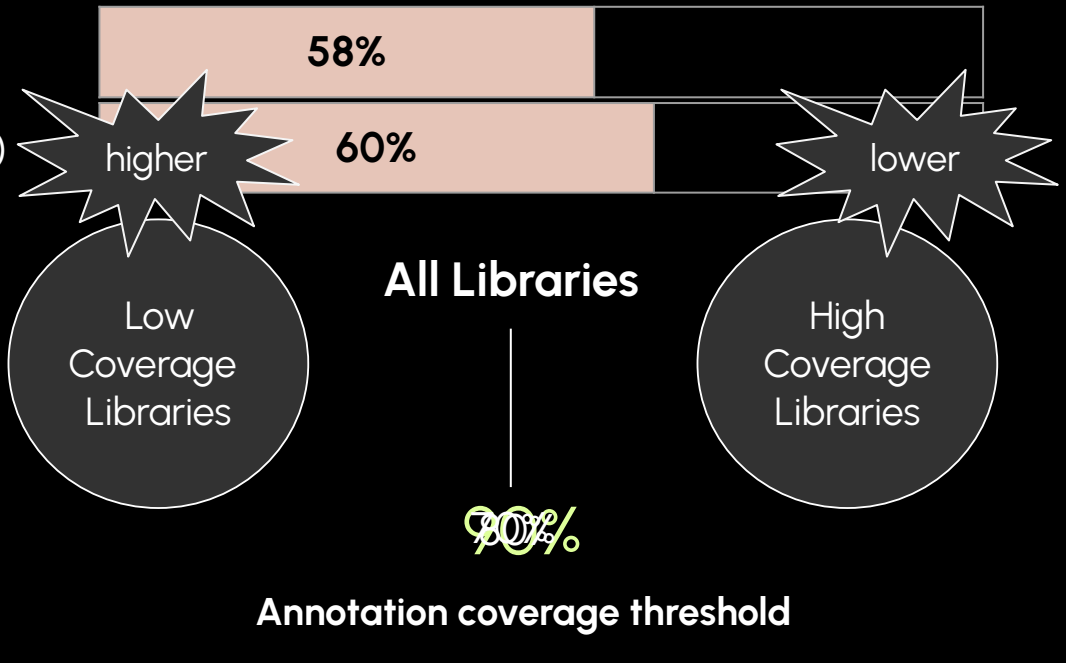
Perhaps not all libraries are equally annotated. Some libraries have only one of their public APIs annotated while some have all annotated.

A library with high annotations coverage  Less Errors

RQ3: Is annotation coverage associated with client-side errors?

(1) **Propagation Rate** (PropRate)

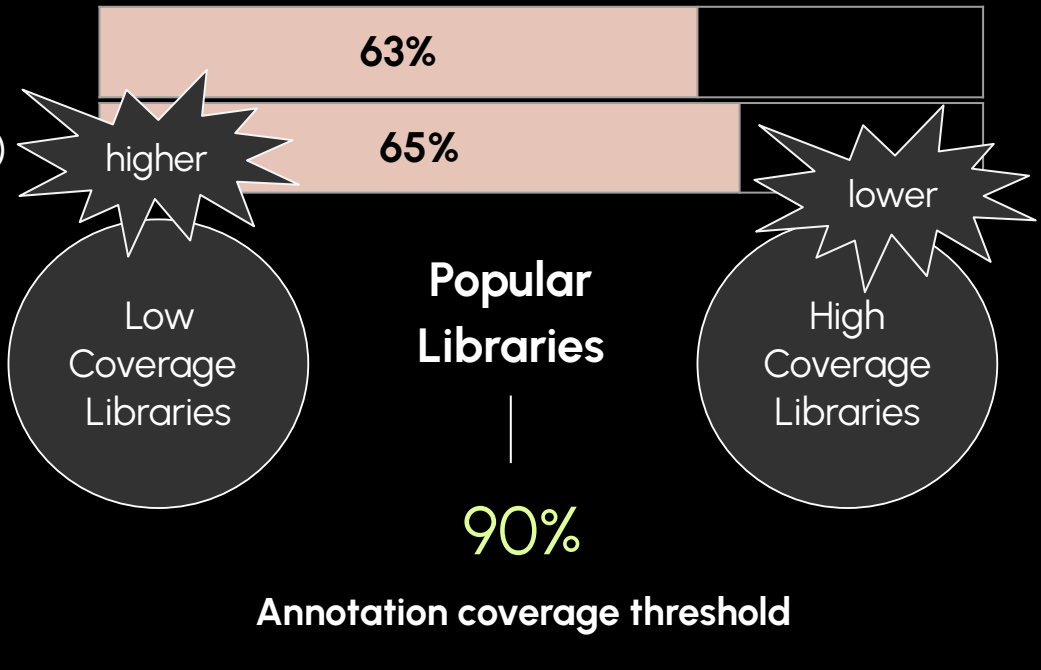
(2) **Median Error Burden** (ErrBurden)



RQ3: Is annotation coverage associated with client-side errors?

(1) **Propagation Rate** (PropRate)

(2) **Median Error Burden** (ErrBurden)



Finding 3: It's an all or nothing situation. Only libraries with >=90% annotation coverage show lower client side errors.

But wait

Higher Annotation (intention and effort)  Client using Type Checker  Less Type Errors (Reap Benefit)

RQ4 Is there an alignment between type checker usage and annotation coverage?

	Client: No Checker	**Client: Uses Checker**
Library Ann: Low <90%	50% aligned (neither side cares)	19% (library missed opportunity)
Library Ann: High >=90%	21% (client missed opportunity)	10% (aligned - ideal)

Finding 4: In 40%, there are mismatches between a library's efforts and its clients. **Only 10% align ideally.**

What we learned so far

Finding 1: 70% of client projects contain at least one LibClientError

Finding 2: Some libraries are indeed more problematic than others, causing more errors in the ecosystem

Finding 3: It's an all or nothing situation. Only libraries with >=90% annotation coverage show lower client side errors.

Finding 4: In 40%, there are mismatches between a library's efforts and its clients. **Only 10% align ideally.**

Implications

Library maintainers can aim for annotation coverage closer to 90%, potentially supported by tools for automated type inference and annotation generation.

As library maintainers annotate more of their APIs, clients should align by adopting type checkers to realize the downstream benefits.



ERIC ASARE
eric.asare@nyu.edu



LUCA DI GRAZIA
work@lucadigrazia.com



SARAH NADI
sarah.nadi@nyu.edu

All or Nothing: How Library Type Annotations Affect Client-Side Errors in Python

Library maintainers can aim for annotation coverage closer to 90%, supported by research on automated type inference and annotation generation.

As library maintainers annotate more of their APIs, clients should align by adopting type checkers to realize the downstream benefits.

Paper

Artifact